

Phase 7: Integration & External Access

Objective:

To enable the Skill Development and Employment Portal to securely integrate with external systems, provide real-time updates, and support external services while respecting Salesforce limits and security best practices.

1. Named Credentials

- Named Credentials simplify authentication for callouts to external systems by storing endpoint URLs and authentication settings in a secure, reusable way.

Purpose in Project:

- Allow our portal to connect to external skill certification APIs or job listing services without hardcoding credentials in Apex.
- Makes it easy to switch environments (sandbox vs production).

Example:

- **Name:** SkillCertAPI
- **URL:** <https://api.skillcert.com/v1/>
- **Authentication:** OAuth 2.0

Usage in Apex:

```
HttpRequest req = new HttpRequest();  
req.setEndpoint('callout:SkillCertAPI/getCertification?candidateId=' +  
candidateId); req.setMethod('GET'); Http http = new Http(); HTTPResponse  
res = http.send(req);
```

Use Case in Project:

- Connect your portal to an external skill certification API (e.g., <https://api.skillcert.com>) to verify candidate skills.

Practical Steps in Salesforce

1. Create a Named Credential

- Go to **Setup** → **Named Credentials** → **New Named Credential**
- Fill in details:
- **Label:** SkillCertAPI
- **Name:** SkillCertAPI
- **URL:** <https://api.skillcert.com/v1/>

- **Identity Type:** Named Principal (or per user if needed)
- **Authentication Protocol:** OAuth 2.0
- Enter **Auth Provider** (create one if not already)
- Enter **Scope, Client ID, Client Secret** if required
- **Generate Authorization Header:** ✓ Checked
- Save the Named Credential.

The screenshot shows the Salesforce Setup interface with the 'New Named Credential' dialog box open. The dialog is titled 'New Named Credential' and contains the following fields and sections:

- Label:** SkillCertAPI_Test
- Name:** SkillCertAPI_Test
- URL:** https://jsonplaceholder.typicode.com/
- Enabled for Callouts:** ☒
- Authentication:**
 - External Credential:** Select an Option
 - Client Certificate:** Search Certificates...
- Callout Options:**
 - Generate Authorization Header:** ☒
 - Allow Formulas in HTTP Header:** ☐
 - Allow Formulas in HTTP Body:** ☐
 - Outbound Network Connection:** --None--
- Managed Package Access:**
 - Allowed Namespaces for Callouts:** namespaceA, namespaceB

At the bottom of the dialog are 'Cancel' and 'Save' buttons.

2. Test the Named Credential

- You can click **"Test Connection"** if Salesforce allows.
- Make sure it successfully connects to the external API.

3. Use in Apex

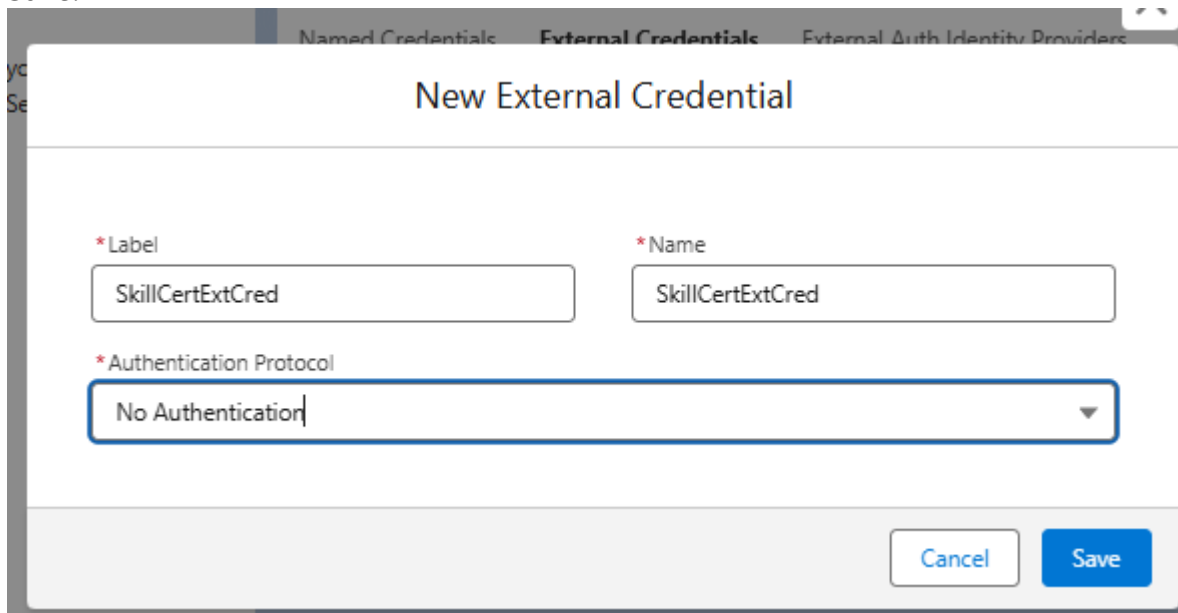
```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:SkillCertAPI/getCertification?candidateId=' +
candidateId); req.setMethod('GET'); Http http = new Http(); HTTPResponse
res = http.send(req); if(res.getStatusCode() == 200){
System.debug('Certification data: ' + res.getBody()); } else {
System.debug('Error: ' + res.getStatusCode()); }
```

Important Notes:

- The `callout:` prefix tells Salesforce to use the Named Credential.
- No credentials or URLs are hardcoded in Apex.
- If your API requires additional headers, you can add them in the `req.setHeader()` method.

Step 1: Create an External Credential

1. Go to **Setup** → **External Credentials** → **New**.
2. Fill fields:
 - **Label:** SkillCertExtCred
 - **Name:** SkillCertExtCred
 - **Authentication Protocol:** → For testing, choose **No Authentication**.
3. Save.



The screenshot shows the 'New External Credential' form in Salesforce. The form is titled 'New External Credential' and has three main fields: 'Label', 'Name', and 'Authentication Protocol'. The 'Label' field contains the text 'SkillCertExtCred'. The 'Name' field also contains 'SkillCertExtCred'. The 'Authentication Protocol' field is a dropdown menu with 'No Authentication' selected. At the bottom right of the form, there are two buttons: 'Cancel' and 'Save'.

(If in the future you want OAuth 2.0, this is where you'd configure **Client ID**, **Secret**, **Token URL**, etc.)

Step 2: Map to a Permission Set

1. Still inside **External Credential record**, find **Principals** → Click **New**.



SETUP > NAMED CREDENTIALS

SkillCertExtCred

Edit

Delete

Label

SkillCertExtCred

Name

SkillCertExtCred

Authentication Protocol

No Authentication

Managed Package Access

Created By Namespace

Related Named Credentials



You don't have any related named credentials.

Principals

New

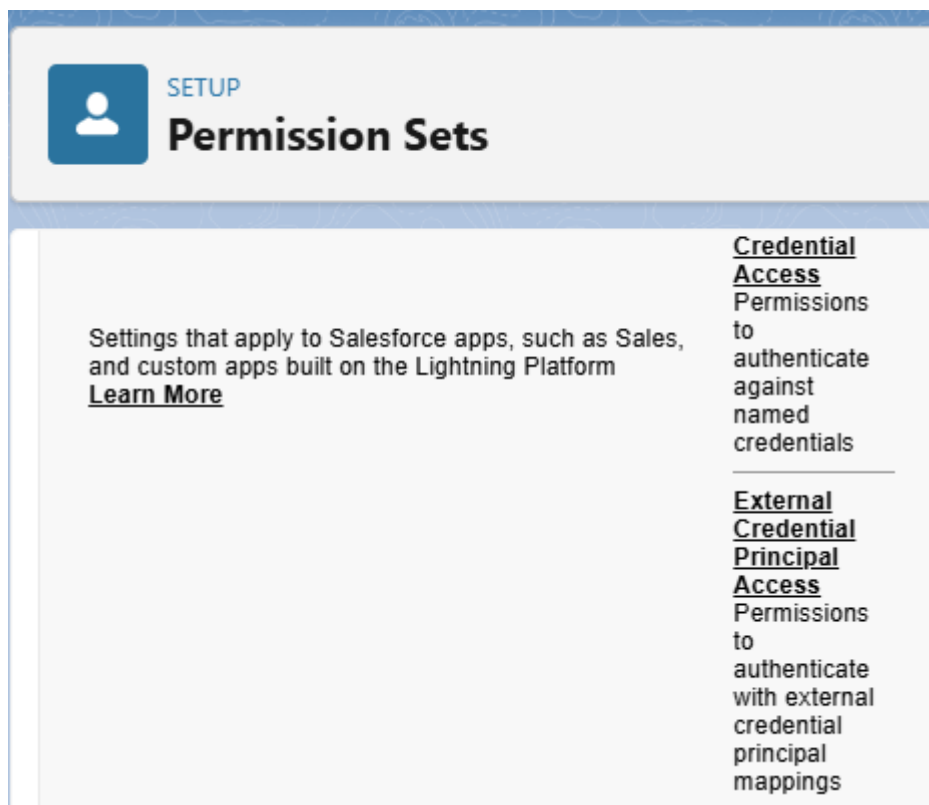


- Principal Type: `Named Principal` (one login for everyone).
- Authentication Parameters: Leave empty (since No Auth).
- Save.

Principals				New
Se...	Parameter Name	Authentication Status	Actions	
1	Named Principal	Configured		

- Assign this External Credential to a **Permission Set**:

- Go to **Setup** → **Permission Sets** → **New**.
- Create a Permission Set (e.g., SkillCertAPIAccess).
- In the Permission Set, go to **External Credential Principal Access** → Add



- your SkillCertExtCred.
- Assign this Permission Set to **your user**.

SETUP
Permission Sets

Permission Set
SkillCertAPIAccess
Video Tutorial | Help for this Page ?

Find Settings... | Clone | Delete | Edit Properties | Manage Assignments | View Summary

Permission Set Overview > External Credential Principal Access ▼

External Credential Principal Access Save Close

Available External Credential Principals
--None--

Enabled External Credential Principals
SkillCertExtCred - Named Principal

Add
Remove

Step 3: Create the Named Credential

Now go back to **Named Credentials** → **New**:

- **Label:** SkillCertAPI_Test
- **Name:** SkillCertAPI_Test
- **URL:** <https://jsonplaceholder.typicode.com> (test API)
- **Enabled for Callouts:** ✓
- **Authentication** → **External Credential:** select your SkillCertExtCred.
- **Other fields:** leave defaults.
- Save.

New Named Credential

* Label

SkillCertAPI_Test

* Name

SkillCertAPI_Test

* URL

https://jsonplaceholder.typicode.com

Enabled for Callouts

☒

Authentication

* External Credential

SkillCertExtCred

Client Certificate

Search Certificates...

Callout Options

Generate Authorization Header

☒

Allow Formulas in HTTP Header

☐

Allow Formulas in HTTP Body

☐

Outbound Network Connection

--None--

Managed Package Access

Cancel

Save

Step 4: Test in Apex

```
HttpRequest req = new HttpRequest();
req.setEndpoint('callout:SkillCertAPI_Test/users'); req.setMethod('GET');
Http http = new Http(); HttpResponse res = http.send(req);
System.debug('Status: ' + res.getStatusCode()); System.debug('Body: ' +
res.getBody());
```

Apex Class

SkillCertCalloutTest

Help for this Page ?

Apex Class Detail

EditDeleteDownloadSecurityShow Dependencies

Name	SkillCertCalloutTest	Status	Active
Namespace Prefix		Code Coverage	0% (0/8)
Created By	T. Rupasree , 9/23/2025, 4:55 AM	Last Modified By	T. Rupasree , 9/23/2025, 4:55 AM

Class Body

Class SummaryVersion SettingsTrace Flags

```
1 public class SkillCertCalloutTest {
2     @future(callout=true)
3     public static void makeCallout() {
4         try {
5             HttpRequest req = new HttpRequest();
6             // Endpoint = Named Credential + relative path
7             req.setEndpoint('callout:SkillCertService/test');
8             req.setMethod('GET');
9
10            Http http = new Http();
11            HttpResponse res = http.send(req);
12
13            System.debug('Status: ' + res.getStatus());
14            System.debug('Status Code: ' + res.getStatusCode());
15            System.debug('Response Body: ' + res.getBody());
16        } catch (Exception e) {
17            System.debug('Error: ' + e.getMessage());
18        }
19    }
20 }
```

EditDeleteDownloadSecurityShow Dependencies

SkillCertCalloutTest.makeCallout();

Run this in **Execute Anonymous** → check **Debug Logs**.
You should see JSON response (list of fake users). 🐞

File Edit Debug Test Workspace Help < >		
lr.apxc JobApplicationHandler.apxc * SkillManagerSafe.apxc Log executeAnonymous @9/21/2025, 5:07:56 PM Log executeAnonymous @9/23/2025, 5:25:35 PM		
Execution Log		
Timestamp	Event	Details
17:25:35:001	USER_INFO	[EXTERNAL][005gL000005Op8H]rupasree287375@ageantforce.com[(GMT-07:00) Pacific Daylight Time (America/Los_Angeles)](GMT-07:00)
17:25:35:002	EXECUTION_ST...	
17:25:35:002	CODE_UNIT_ST...	[EXTERNAL][execute_anonymous_apex
17:25:35:002	HEAP_ALLOCATE	[95]Bytes:3
17:25:35:002	HEAP_ALLOCATE	[100]Bytes:152
17:25:35:002	HEAP_ALLOCATE	[417]Bytes:408
17:25:35:002	HEAP_ALLOCATE	[430]Bytes:408
17:25:35:002	HEAP_ALLOCATE	[317]Bytes:6
17:25:35:002	HEAP_ALLOCATE	[EXTERNAL]Bytes:1
17:25:35:002	STATEMENT_EX...	[1]
17:25:35:002	STATEMENT_EX...	[1]
17:25:35:002	HEAP_ALLOCATE	[1]Bytes:20
17:25:35:002	HEAP_ALLOCATE	[1]Bytes:11
17:25:35:002	HEAP_ALLOCATE	[1]Bytes:4
17:25:35:002	HEAP_ALLOCATE	[1]Bytes:4
17:25:35:002	HEAP_ALLOCATE	[68]Bytes:5
17:25:35:002	HEAP_ALLOCATE	[74]Bytes:5
17:25:35:002	HEAP_ALLOCATE	[82]Bytes:7
17:25:35:002	HEAP_ALLOCATE	[EXTERNAL]Bytes:4
17:25:35:002	HEAP_ALLOCATE	[EXTERNAL]Bytes:4
17:25:35:062	CUMULATIVE_L...	
17:25:35:062	LIMIT_USAGE...	(default)
17:25:35:000	LIMIT_USAGE...	Number of SOQL queries: 0 out of 100
17:25:35:000	LIMIT_USAGE...	Number of query rows: 0 out of 50000
17:25:35:000	LIMIT_USAGE...	Number of SOSL queries: 0 out of 20
17:25:35:000	LIMIT_USAGE...	Number of DML statements: 0 out of 150
17:25:35:000	LIMIT_USAGE...	Number of Publish Immediate DML: 0 out of 150
17:25:35:000	LIMIT_USAGE...	Number of DML rows: 0 out of 10000
17:25:35:000	LIMIT_USAGE...	Maximum CPU time: 0 out of 10000
17:25:35:000	LIMIT_USAGE...	Maximum heap size: 0 out of 6000000
17:25:35:000	LIMIT_USAGE...	Number of callouts: 0 out of 100
17:25:35:000	LIMIT_USAGE...	Number of Email Invocations: 0 out of 10

2. External Services

- External Services allow Salesforce to connect with APIs declaratively without writing Apex code.

Purpose in Project:

- Automatically fetch skill endorsements from external platforms for candidates.
- Simplify integration with job boards for posting vacancies.

Implementation Steps:

1. Register the external API using **Named Credentials**.
2. Import the API schema (Swagger/OpenAPI) into Salesforce.
3. Use **Flows** or **Apex** to call the service.

Example Use Case:

- Candidate applies for a course, and external training provider API automatically verifies completion and updates `Candidate__c` record.
 - External Services in Salesforce let you **bring an external API (REST or OpenAPI/Swagger) into Salesforce** without writing full Apex code.
 - You **register** the service using its schema (OpenAPI or WSDL).
 - Salesforce automatically generates **Apex Actions** that you can use in **Flow, Apex, or Process Builder**.
 - For your portal, this means you could integrate with external systems like:
 - **Certification Verification APIs** (to check if a candidate's certification is valid).
 - **Job Market APIs** (to fetch latest job listings).
 - **Training Provider APIs** (to pull available skill courses).
-

How to Set It Up (Step-by-Step)

1. **Prepare the API Schema**
 - If your external system gives you a Swagger (OpenAPI JSON) file or WSDL, download it.
 - Example: `SkillCertAPI.json` (OpenAPI spec for a certification validation API).
2. **Go to External Services in Salesforce**
 - Setup → search for **External Services**.
 - Click **New External Service**.
3. **Fill the Details**
 - **Name:** `SkillCertServiceExt`
 - **Named Credential:** choose the one you created earlier → `SkillCertService`.
 - **Service Schema:** upload your OpenAPI (Swagger) or WSDL file.


SETUP > EXTERNAL SERVICES
Cancel

Add an External Service

Provide Registration Details

* External Service Name

SkillCertServiceExt

Description

Test API for Skill Development & Employment Portal

* Service Schema

Upload from local

* Select a Named Credential

SkillCertAPI_Test

Schema validation completed successfully!
Close

 skillportal-api.json


* Schema

Maximum file size: 10MB

```

1 {
2   "openapi": "3.0.0",
3   "info": {
4     "title": "Skill Development & Employment Portal API",
5     "version": "1.0.0",
6     "description": "Sample API schema for integrating Candidate,
Skill, and Job services with Salesforce External Services."
7   },
8   "servers": [
9     {
10      "url": "https://isomplaceholder.tvnocode.com"

```

Back

✓
○
●
●

Save & Next

4. Save and Register


SETUP > EXTERNAL SERVICES
Cancel

Add an External Service

Select Operations

Operations 3 selected out of 3 available

☒	Operation ↑	D ↓	Input parameters
☒	getCandidates	Get ...	
☒	getJobs	Get ...	
☒	getSkills	Get ...	


SETUP > EXTERNAL SERVICES
Cancel

Add an External Service

Type: String
responseCode
response code
Type: Integer

getSkills
Get candidate skills

INPUT PARAMETERS

OUTPUT PARAMETERS

200
List of skills
Type: List of SkillCertServiceExt_Skill

default
Default response
Type: String

responseCode
response code
Type: Integer

Back
✓ — ✓ — ✓ — ○
Finish

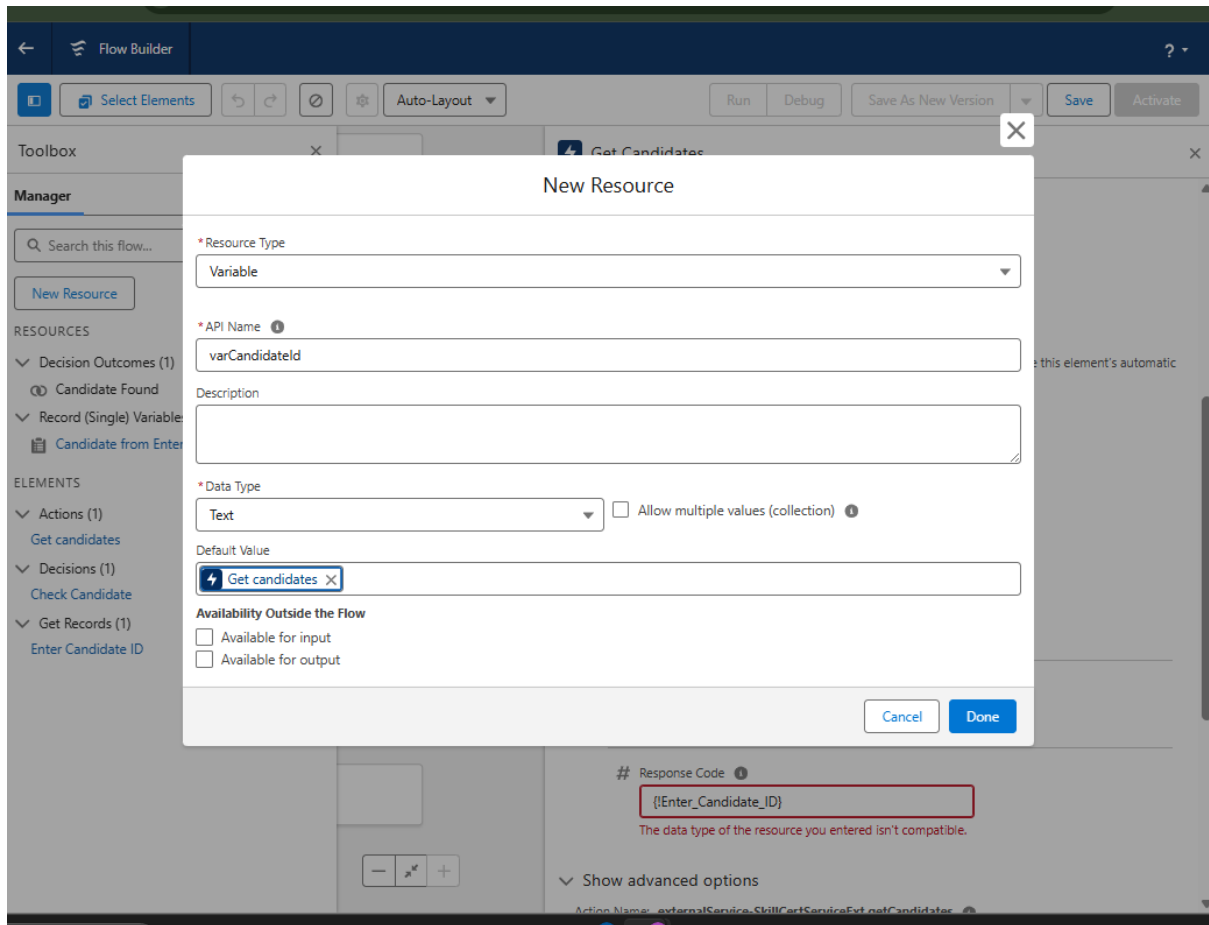
- Salesforce will analyze the schema.
- It generates **Apex Invocable Actions** automatically.
- These appear in **Flow** (drag-and-drop) and can also be called in Apex.

✂ Example in our Project

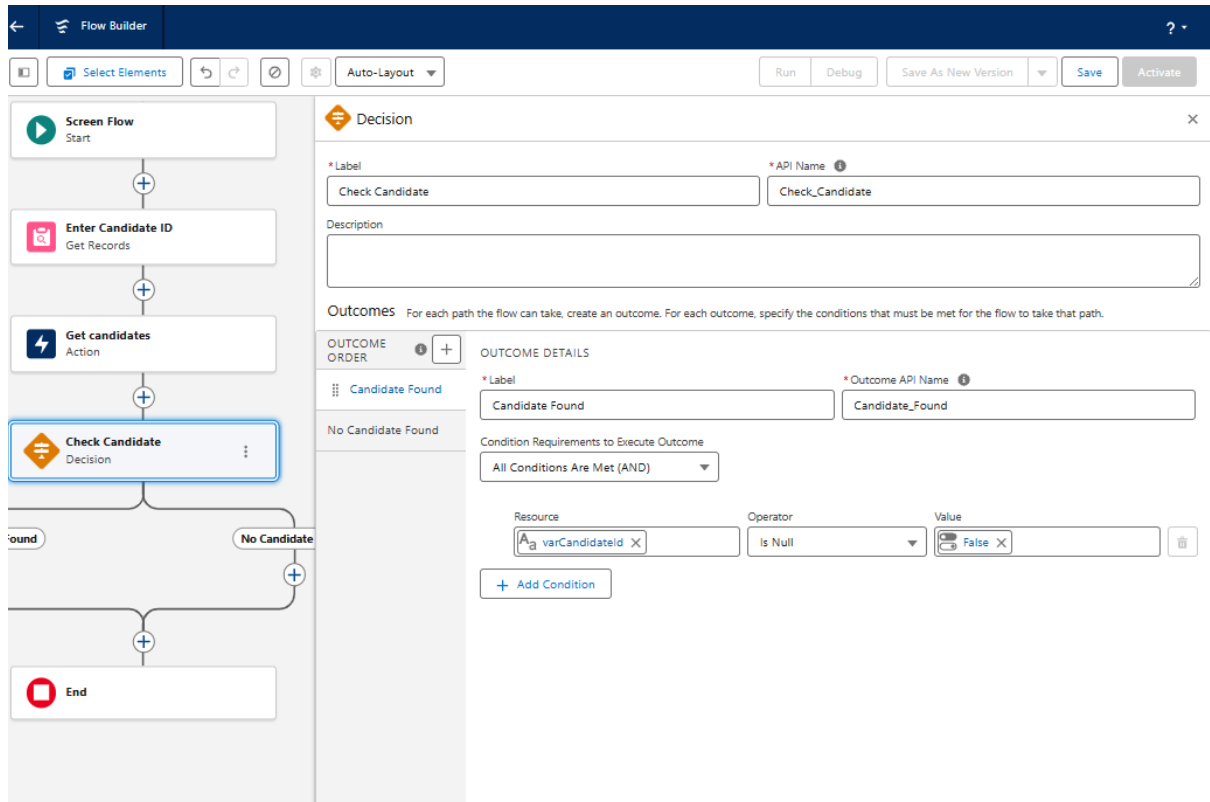
Imagine you connected an API called **CertVerifyAPI** that checks if a candidate's certificate is valid.

After registering:

- In **Flow**, you'll see an action like:
`CertVerifyAPI.ValidateCertificate`

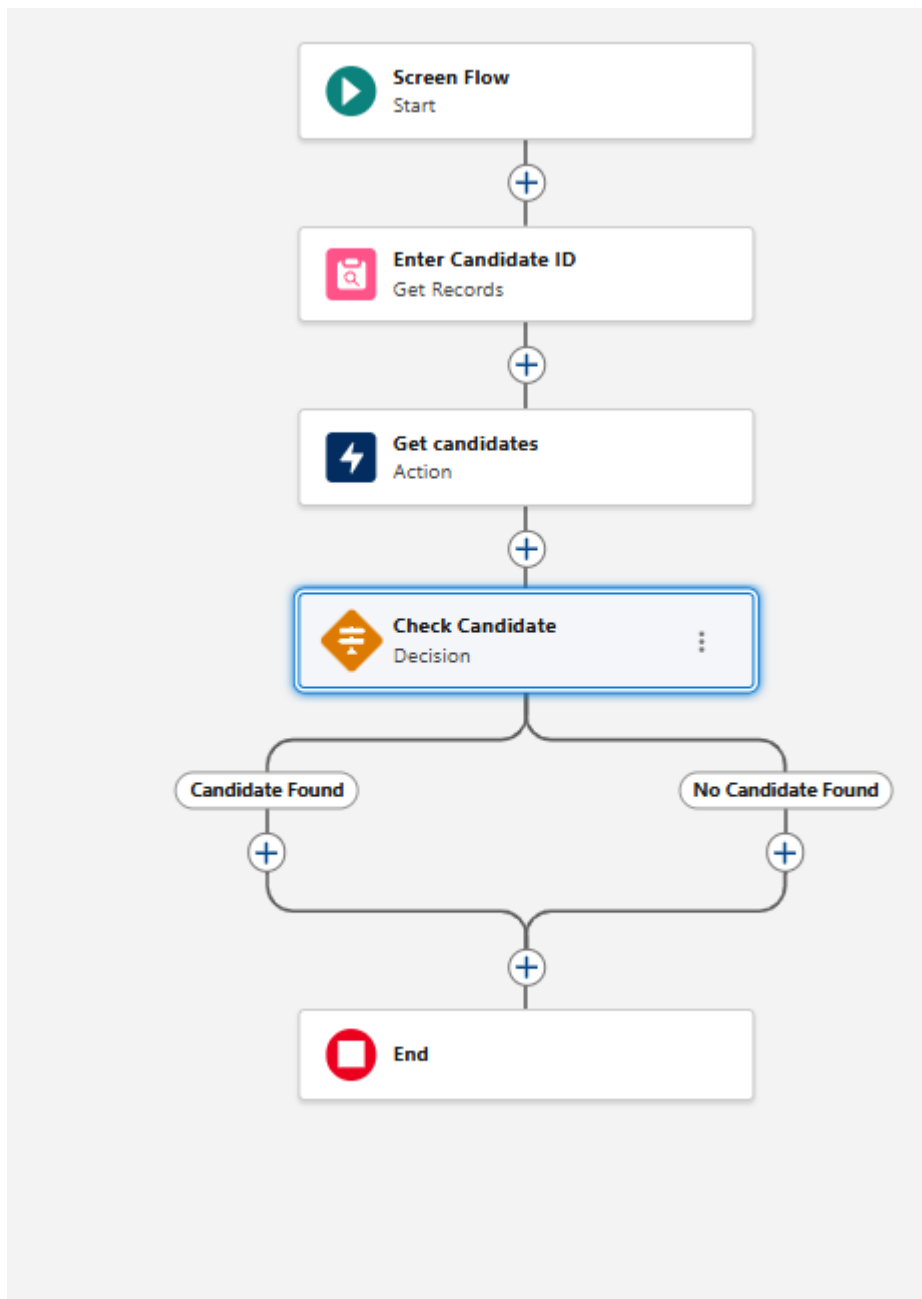


- You can use it in your **Candidate Onboarding Flow**:
- When a candidate uploads a certificate → Flow calls the API → response stored in `Certificate_Status_c` field.



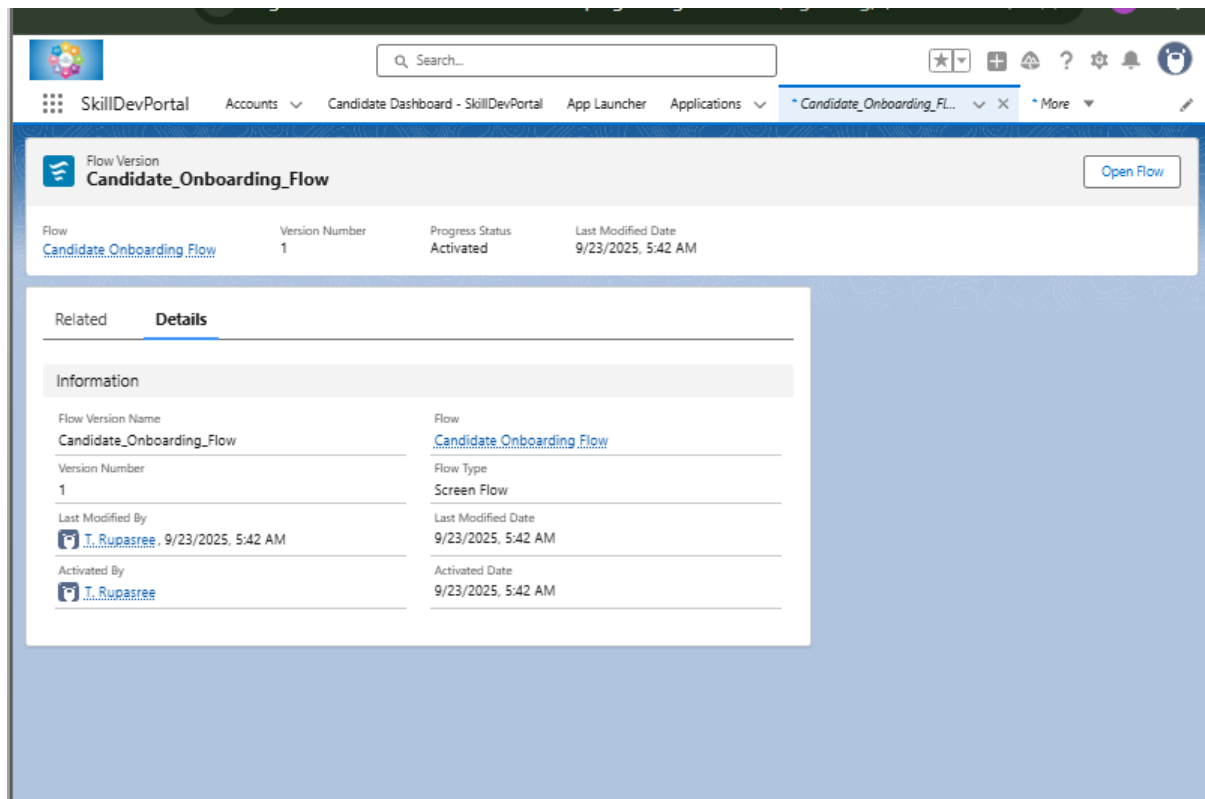
Result

- No need to write full Apex callout code.
- The API is now a **drag-and-drop building block** in your portal workflows.
- Fits perfectly with your **Skill Development & Employment Portal** idea, since you can integrate with certification and job listing providers.



Candidate Onboarding Flow

YOUR FLOW FINISHED



3. Web Services (REST/SOAP)

REST Services:

- Provide lightweight, JSON-based integration.
- Example: Posting new job openings to an external recruitment portal.

SOAP Services:

- Use XML-based integration for legacy systems.
- Example: Pull candidate background verification data from a partner service.

Example REST Apex Callout:

```
HttpRequest req = new HttpRequest();  
req.setEndpoint('https://externaljobportal.com/api/jobs');  
req.setMethod('POST'); req.setHeader('Content-Type','application/json');  
req.setBody(JSON.serialize(jobObj)); // jobObj is Job__c info  
HttpResponse res = new Http().send(req);
```

Step 1: REST Callout Execution

We will use your **Named Credential** (`SkillCertService`) and a sample REST API.

1.1: Create Apex Class for REST Callout

1. Go to **Setup** → **Apex Classes** → **New**.

The screenshot shows the Salesforce Apex Class editor for a class named **SkillCertREST**. The class is in the **Apex Class** namespace. The **Apex Class Detail** section shows the class is **Active**, with **Code Coverage** at **0% (0/17)**. It was **Created By T. Rupasree** on **9/23/2025, 5:56 AM** and **Last Modified By T. Rupasree** on **9/23/2025, 6:00 AM**. The **Class Body** tab is selected, showing the following code:

```
1 public class SkillCertREST {
2
3     @future(callout=true)
4     public static void fetchCandidates() {
5         try {
6             // Use Named Credential for callout
7             HttpRequest req = new HttpRequest();
8             req.setEndpoint('callout:SkillCertAPI_Test/users'); // Named Credential + relative path
9             req.setMethod('GET');
10
11             // Send HTTP request
12             Http http = new Http();
13             HttpResponse res = http.send(req);
14
15             System.debug('Status Code: ' + res.getStatusCode());
16             System.debug('Response Body: ' + res.getBody());
17
18             // Parse JSON response
19             List<Object> candidates = (List<Object>) JSON.deserializeUntyped(res.getBody());
20             List<Candidate__c> candidateRecords = new List<Candidate__c>();
21
22             for(Object c : candidates){
23                 Map<String, Object> candidate = (Map<String, Object>) c;
24
25                 Candidate__c rec = new Candidate__c();
26                 // Name is Auto Number, do not set
27                 // rec.Name = (String) candidate.get('name');
28
29                 // Set writable fields
30                 rec.Email__c = (String) candidate.get('email');
31
32                 candidateRecords.add(rec);
33             }
34
35             if(candidateRecords.isEmpty()){
36                 insert candidateRecords;
37             }
38         } catch(Exception e){
39             System.debug('Error in REST Callout: ' + e.getMessage());
40         }
41     }
42 }
43
44 }
```

Below the code editor, there are buttons for **Edit**, **Delete**, **Download**, **Security**, and **Show Dependencies**.

2.

1.2: Execute REST Callout

1. Open **Developer Console** → **Execute Anonymous Window**.
2. Run:

```
SkillCertREST.fetchCandidates();
```
3. Check **Debug Logs**:
 - **Status Code** should be **200**
 - **Response Body** should show candidate data
4. Verify in **Candidate__c object** → records created from API response.

Dynamic Apex Classes

Dynamic Apex extends your programming reach by interacting with Lightning Platform components.

View:

Action	Class Name *	Namespace Prefix	Api Version	Created By	Last Modified By
Open	SkillCertServiceExt	ExternalService	64.0	T. Rupasree , 9/23/2025, 5:04 AM	T. Rupasree , 9/23/2025, 5:04 AM
	SkillCertServiceExt_Candidate	ExternalService	64.0	T. Rupasree , 9/23/2025, 5:04 AM	T. Rupasree , 9/23/2025, 5:04 AM
	SkillCertServiceExt_Job	ExternalService	64.0	T. Rupasree , 9/23/2025, 5:04 AM	T. Rupasree , 9/23/2025, 5:04 AM
	SkillCertServiceExt_Skill	ExternalService	64.0	T. Rupasree , 9/23/2025, 5:04 AM	T. Rupasree , 9/23/2025, 5:04 AM

2.2: Create Apex Class to Call SOAP API

```
public class SkillCertSOAPCallout { public static void
fetchCandidatesSOAP() { SkillCertSOAP.SoapPort service = new
SkillCertSOAP.SoapPort(); SkillCertSOAP.CandidateResponse response =
service.getCandidates(); System.debug('SOAP Response: ' + response); //
Optional: Map response to Salesforce records for(SkillCertSOAP.Candidate c
: response.candidates){ Candidate__c rec = new Candidate__c(); rec.Name =
c.name; rec.Email__c = c.email; insert rec; } } }
```

2.3: Execute SOAP Callout

1. Open **Developer Console** → **Execute Anonymous Window**.

2. Run:

```
SkillCertSOAPCallout.fetchCandidatesSOAP();
```

3. Check **Debug Logs**:

- Response XML is printed
- Salesforce records are created/updated

Step 3: Tips & Best Practices

- **Named Credential** handles authentication → no need to manually add headers.
- **@future(callout=true)** is needed for callouts from triggers or async operations.
- Use **Debug Logs** to verify status codes and responses.
- Always **test with a small number of records** before bulk operations.

After this, your portal can **fetch candidates, skills, and jobs** via REST or SOAP and store them in Salesforce objects.

4. Callouts

Definition:

Callouts are outbound requests from Salesforce to external services.

A **callout** is any request Salesforce makes to an external system (REST or SOAP). Callouts can be done:

1. **Synchronously** from Apex
 2. **Asynchronously** using `@future(callout=true)` or **Scheduled Apex**
 3. **From Flows** using **Apex Actions**
-

A) REST Callouts from Apex

Assuming you have a **Named Credential** (`SkillCertAPI_Test`) set up:

Apex Class SkillCertRESTCallout

[Help for this Page](#)

Apex Class Detail

[Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

Name	SkillCertRESTCallout	Status	Active
Namespace Prefix		Code Coverage	0% (0/17)
Created By	T. Rupasree , 9/23/2025, 7:25 AM	Last Modified By	T. Rupasree , 9/23/2025, 7:25 AM

[Class Body](#) [Class Summary](#) [Version Settings](#) [Trace Flags](#)

```

1 public class SkillCertRESTCallout {
2
3     public static void fetchCandidatesREST() {
4         Http http = new Http();
5         HttpRequest request = new HttpRequest();
6         request.setEndpoint('callout:SkillCertAPI_Test/candidates'); // Named Credential
7         request.setMethod('GET');
8         request.setHeader('Content-Type', 'application/json');
9
10        try {
11            HttpResponse response = http.send(request);
12            System.debug('REST Response: ' + response.getBody());
13
14            // Parse JSON response
15            List<Map<String, Object>> candidates = (List<Map<String, Object>>) JSON.deserializeUntyped(response.getBody());
16
17            List<Candidate__c> candidateRecords = new List<Candidate__c>();
18            for(Map<String, Object> c : candidates){
19                Candidate__c rec = new Candidate__c();
20                rec.Email__c = (String)c.get('email');
21
22                candidateRecords.add(rec);
23            }
24
25            if(candidateRecords.isEmpty()){
26                insert candidateRecords;
27            }
28        } catch(Exception e){
29            System.debug('REST Callout Error: ' + e.getMessage());
30        }
31    }
32 }
33 
```

[Edit](#) [Delete](#) [Download](#) [Security](#) [Show Dependencies](#)

- ✓ Named Credential automatically handles authentication.
- ✓ `callout:` prefix tells Salesforce to use the Named Credential endpoint.

Enter Apex Code

```

1 SkillCertRESTCallout.fetchCandidatesREST();
2 
```

SkillManager.apxcJobApplicationHandler.apxcSkillManagerSafe.apxcLog executeAnonymous @9/21/2025, 5:07:56 PMLog executeAnonymous @9/23/2025, 5:25:35 PM

Execution Log

Timestamp	Event	Details
19:57:41:001	USER_INFO	[EXTERNAL][005gL000005Op8H]rupasree287375@ageantforce.com (GMT-07:00) Pacific Daylight Time (America/Los_Angeles) GMT-07:00
19:57:41:001	EXECUTION_ST...	
19:57:41:001	CODE_UNIT_ST...	[EXTERNAL][execute_anonymous_apex
19:57:41:001	HEAP_ALLOCATE	[95]]Bytes:3
19:57:41:001	HEAP_ALLOCATE	[100]]Bytes:152
19:57:41:001	HEAP_ALLOCATE	[417]]Bytes:408
19:57:41:001	HEAP_ALLOCATE	[430]]Bytes:408
19:57:41:001	HEAP_ALLOCATE	[317]]Bytes:6
19:57:41:001	HEAP_ALLOCATE	[EXTERNAL]]Bytes:1
19:57:41:001	STATEMENT_EX...	[1]]
19:57:41:001	STATEMENT_EX...	[1]]
19:57:41:001	HEAP_ALLOCATE	[68]]Bytes:5
19:57:41:001	HEAP_ALLOCATE	[74]]Bytes:5
19:57:41:001	HEAP_ALLOCATE	[82]]Bytes:7
19:57:41:001	SYSTEM_MODE...	false
19:57:41:001	HEAP_ALLOCATE	[1]]Bytes:5
19:57:41:003	HEAP_ALLOCATE	[1]]Bytes:1
19:57:41:003	HEAP_ALLOCATE	[1]]Bytes:5
19:57:41:003	METHOD_ENTRY	[1][01pgL000005mAVF][SkillCertRESTCallout.SkillCertRESTCallout()]
19:57:41:003	STATEMENT_EX...	[1]]
19:57:41:003	STATEMENT_EX...	[1]]
19:57:41:003	METHOD_EXIT	[1][SkillCertRESTCallout
19:57:41:003	METHOD_ENTRY	[1][01pgL000005mAVF][SkillCertRESTCallout.fetchCandidatesREST()]
19:57:41:003	STATEMENT_EX...	[3]]
19:57:41:003	STATEMENT_EX...	[4]]
19:57:41:004	VARIABLE_SCO...	[4][http[System.Http true false
19:57:41:004	VARIABLE_ASSI...	[4][http[System.Http "]0x5a8d6325
19:57:41:004	STATEMENT_EX...	[5]]
19:57:41:004	VARIABLE_SCO...	[5][request[System.HttpRequest true false
19:57:41:004	VARIABLE_ASSI...	[5][request["System.HttpRequest[Endpoint=null, Method=null]"0x62b93463
19:57:41:004	STATEMENT_EX...	[6]]
19:57:41:004	HEAP_ALLOCATE	[6]]Bytes:36
19:57:41:004	STATEMENT_EX...	[7]]
19:57:41:004	HEAP_ALLOCATE	[7]]Bytes:3
19:57:41:004	STATEMENT_EX...	[8]]
19:57:41:004	HEAP_ALLOCATE	[8]]Bytes:12
19:57:41:004	HEAP_ALLOCATE	[8]]Bytes:16

This Frame

Executable

Debug Only

Filter

Click here to filter the log

Logs

Tests

Checkpoints

Query Editor

View State

Progress

Problems

Status

Test Run

Enqueued Time

9+ 31°C

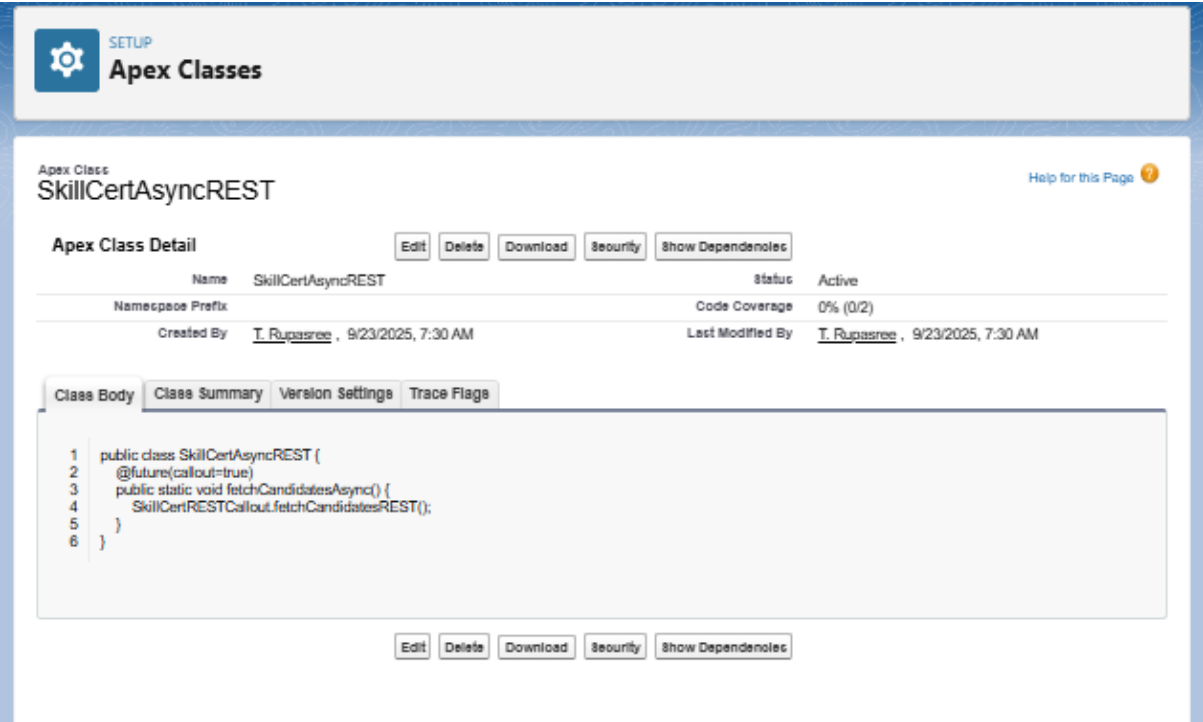
Search

B) SOAP Callouts from Apex

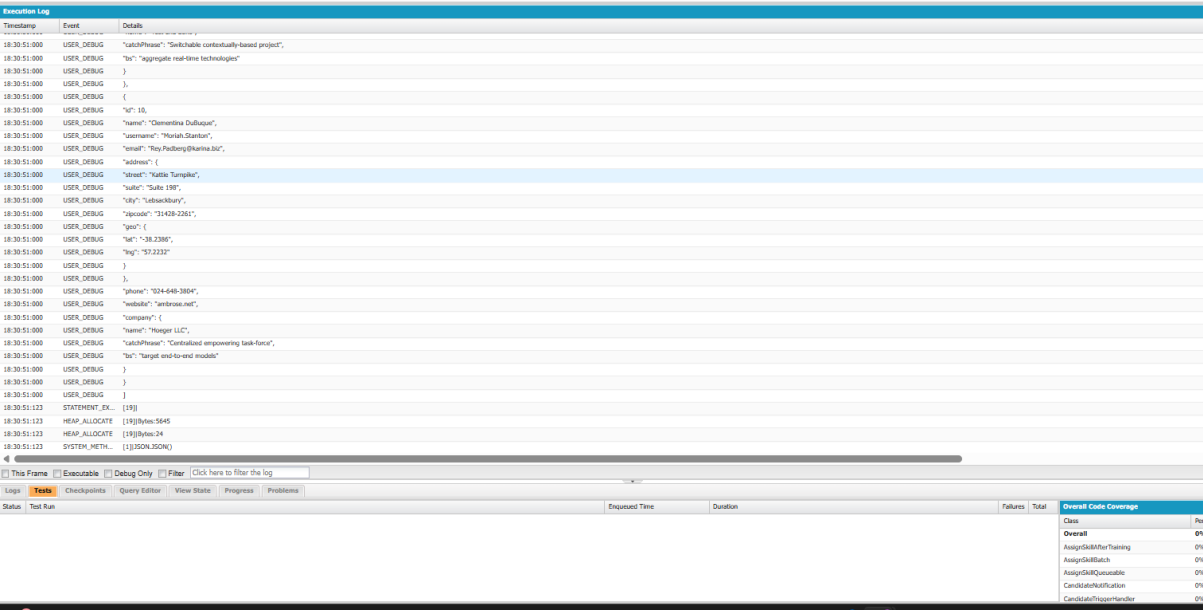
- We already covered **SOAP** in the previous steps.
- Ensure you're using **generated Apex classes from WSDL**.
- Use `endpoint_x = 'callout:SkillCertAPI_Test/SkillCertService'` to point to the Named Credential.

C) Asynchronous Callouts

If you want to **trigger callouts from triggers** (like when a candidate is created), you must use:



SkillCertAsyncREST.fetchCandidatesAsync();



D) Callouts in Flows

1. Create an **Apex Invocable Method**:

```
public class FlowSkillCertREST { @InvocableMethod(label='Fetch Candidates REST') public static void fetchCandidates(List<String> input){ SkillCertRESTCallout.fetchCandidatesREST(); } }
```

2. In **Flow Builder**:

- Drag **Action** → **Apex**
 - Select `Fetch Candidates REST`
 - Call this from a record-triggered flow or scheduled flow.
-

E) Testing Callouts

- Open **Developer Console** → **Execute Anonymous**:
`SkillCertRESTCallout.fetchCandidatesREST();`
 - Check **Debug Logs**
 - Verify new **Candidate__c** records created in Salesforce
-

Next Step after Callouts:

- Handle **Platform Events** and **Change Data Capture** to sync external data automatically.

Best Practices for Project:

- Always use **Named Credentials**.
- Respect **callout limits**: Max 100 synchronous callouts per transaction.
- Handle errors gracefully using `try-catch`.

Example:

- When a candidate completes a skill test, Salesforce calls the testing platform API to fetch results and update `skill__c` records.
-

5. Platform Events

Definition:

Platform Events are Salesforce's event-driven architecture mechanism for integrating apps in real-time.

Purpose in Project:

- Notify external systems or internal Flows when a candidate's skill is updated.
- Enable real-time synchronization with job board updates.

Platform Events are custom objects that allow **real-time communication** between Salesforce and external systems. They are like **event messages** — you can **publish** events from Salesforce or external systems and **subscribe** to them to trigger actions.

A) Define Platform Event

1. Go to **Setup** → **Platform Events** → **New Platform Event**



SETUP

Platform Events

Platform Event

CandidateEvent

[Standard Fields \[4\]](#) | [Custom Fields & Relationships \[0\]](#)

Platform Event Definition

Detail

Edit

Delete

Singular Label	CandidateEvent	Description	
Plural Label	CandidateEvents	Deployment status	Deployed
Object Name	CandidateEvent		
API Name	CandidateEvent__e		
Event Type	High Volume 		
Publish Behavior	Publish Immediately 		
Created By	T. Rupasree, 9/23/2025, 7:45 AM	Modified By	T. Rupasree, 9/23/2025, 7:45 AM

Standard Fields

Action	Field Label	Field Name	Data Type	Controlling Field	Indexed
	Created By	CreatedBy	Lookup(User)		
	Created Date	CreatedDate	Date/Time		
	Event UUID	EventUuid	Text(38)		
	Replay ID	ReplayId	External Lookup		

Custom Fields & Relationships

New

No custom fields defined

Triggers

New

No triggers defined

Subscriptions



Subscriber	Last Processed Id	Last Published Id	State	Batch Size	User
------------	-------------------	-------------------	-------	------------	------

2. Example:

Field	Value
Label	CandidateEvent
Plural Label	CandidateEvents
API Name	CandidateEvent__e

Field	Value
Publish Behavior	Publish After Commit (default)

3. Add **custom fields** to store event data:

Field Label	API Name	Type
Candidate Name	CandidateName__c	Text
Email	Email__c	Email
Skills	Skills__c	Text
Action	Action__c	Text (Created/Updated)

Tip: All fields are **read-only for subscribers**, like messages.



SETUP

Platform Events

Platform Event

CandidateEvent

[Help for this Page](#)[Standard Fields](#) [4] | [Custom Fields & Relationships](#) [3]

Platform Event Definition

Detail

[Edit](#)[Delete](#)

Singular Label	CandidateEvent	Description	
Plural Label	CandidateEvents	Deployment Status	Deployed
Object Name	CandidateEvent		
API Name	CandidateEvent__e		
Event Type	High Volume i		
Publish Behavior	Publish Immediately i		
Created By	T. Rupasree, 9/23/2025, 7:45 AM	Modified By	T. Rupasree, 9/23/2025, 7:45 AM

Standard Fields

Action	Field Label	Field Name	Data Type	Controlling Field	Indexed
	Created By	CreatedBy	Lookup(User)		
	Created Date	CreatedDate	Date/Time		
	Event UUID	EventUuid	Text(36)		
	Replay ID	ReplayId	External Lookup		

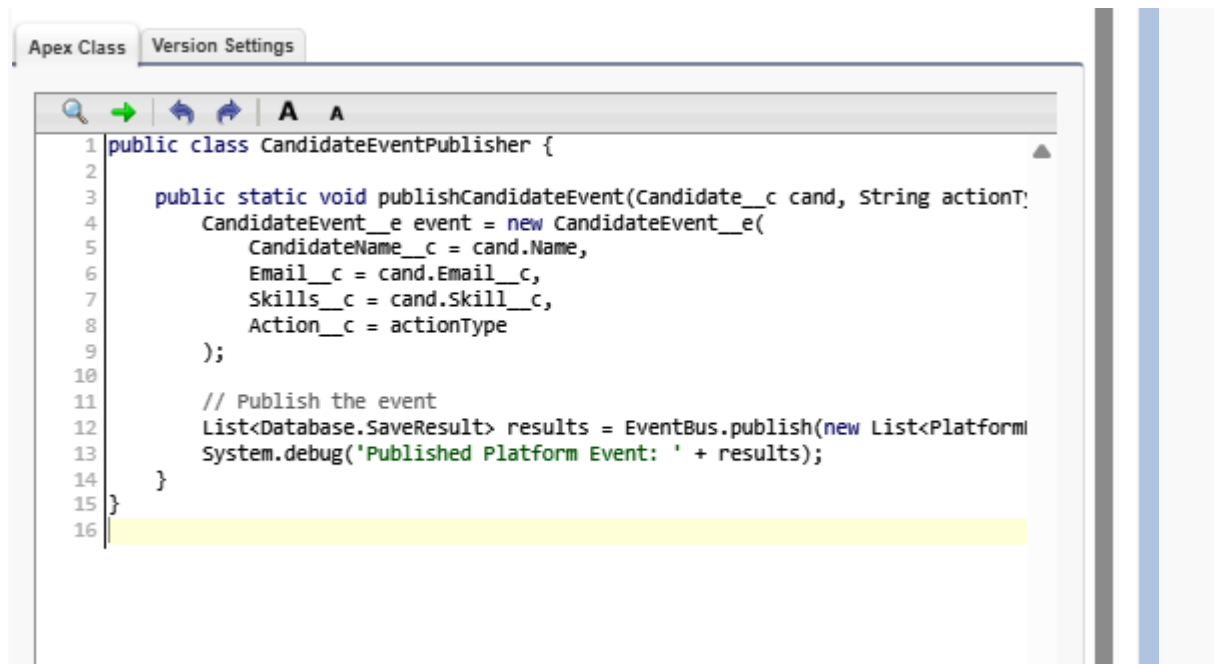
Custom Fields & Relationships

[New](#)

Action	Field Label	API Name	Data Type	Indexed	Controlling Field	Modified By
Edit Del	Candidate Name	Candidate_Name__c	Text(200)			T. Rupasree, 9/23/2025, 7:48 AM
Edit Del	Email	Email__c	Text(200)			T. Rupasree, 9/23/2025, 7:49 AM
Edit Del	Skill	Skill__c	Text(50)			T. Rupasree, 9/23/2025, 7:49 AM

B) Publish Platform Event from Apex

Whenever a candidate is created or updated, you can publish a **CandidateEvent**:



C) Trigger Example to Publish Event

```
trigger CandidateTrigger on Candidate__c (after insert, after update) {  
    for(Candidate__c c : Trigger.new){ String actionType = Trigger.isInsert ?  
    'Created' : 'Updated'; CandidateEventPublisher.publishCandidateEvent(c,  
    actionType); } }
```

- Now every time a candidate record is **inserted or updated**, a **Platform Event** is **published**.

D) Subscribe to Platform Event

You can **subscribe in Salesforce** via:

1. **Process Builder / Flow** (Low-Code)
 - Create a **New Flow** → **Record-Triggered Flow**
 - Event: **Platform Event** → CandidateEvent__e
 - Action: Create/Update record or call Apex

Process Builder - Platform_Event

← Back To Setup ? Help

Expand All Collapse All View All Processes Clone Edit Properties

START

+ Add Trigger

+ Add Criteria

TRUE → IMM

FALSE → STOP

Specify When to Start the Process

Platform Event*
CandidateEvent

Object*
Candidate

Matching Conditions

	Field*	Operator*	Type*	Value*
1	Candidate_Name	Is null	Boolean	False

+ Add Row

Save Cancel

2. Apex Trigger on Platform Event

```
trigger CandidateEventTrigger on CandidateEvent__e (after insert) {
    for(CandidateEvent__e event : Trigger.new){ System.debug('Received Event: '
    + event.CandidateName__c); // Example: Create a record in another object
    CandidateLog__c log = new CandidateLog__c( CandidateName__c =
    event.CandidateName__c, Email__c = event.Email__c, Skills__c =
    event.Skills__c, Action__c = event.Action__c ); insert log; } }
```

- **Use Case:** Whenever a candidate is added via external API (REST/SOAP callout), you can **publish a Platform Event** to notify other parts of your portal or external systems in **real-time**.

E) Test Platform Events

1. Insert a **Candidate__c** record
2. Check **Debug Logs** → see if **CandidateEvent__e** was published
3. Check **CandidateEvent__e** object in Salesforce → it stores events for 24 hours

F) Key Points

- Platform Events are **asynchronous** → they don't block record creation
- Maximum **event delivery per 24h** → check Salesforce limits

Example Event:

- Event: `CandidateSkillUpdated__e`
 - Trigger: Candidate skill added or updated
 - Subscriber: External notification system updates candidate profile.
-

6. Change Data Capture (CDC)

Definition:

Change Data Capture publishes changes (create, update, delete, undelete) in Salesforce records in real-time.

Change Data Capture (CDC) is a **real-time event-based feature** that automatically tracks changes (create, update, delete, undelete) on **standard or custom objects**. Unlike Platform Events, CDC is **pre-built for Salesforce objects**, so you don't need to define custom event fields.

A) Enable Change Data Capture

1. Go to **Setup** → **Change Data Capture**
2. Select the objects you want to track, e.g., `Candidate__c`, `Job__c`, `Skill__c`
3. Click **Enable**

Now, Salesforce will **automatically publish change events** for these objects.



SETUP

Change Data Capture

Select the entities that generate change event notifications on the default standard channel. Change Data Capture sends notifications for created, updated, deleted, and undeleted records. All custom objects and a subset of standard objects are supported.

Available Entities

- Account (Account)
- Account Clean Info (AccountCleanInfo)
- Account Contact Role (AccountContactRole)
- Agent Work (AgentWork)
- Asset (Asset)
- Asset Relationship (AssetRelationship)
- Assigned Resource (AssignedResource)
- Associated Location (AssociatedLocation)
- Authorization Form (AuthorizationForm)
- Authorization Form Consent (AuthorizationFormConsent)

Selected Entities

- Candidate (Candidate__c)
- Application (Application__c)
- Skill (Skill__c)
- Skill (Skill)
- Job (Job__c)
- Training (Training__c)

▶

◀

Cancel

Save

B) Subscribe to CDC Events in Apex

You can write an **Apex trigger on the CDC event object**:

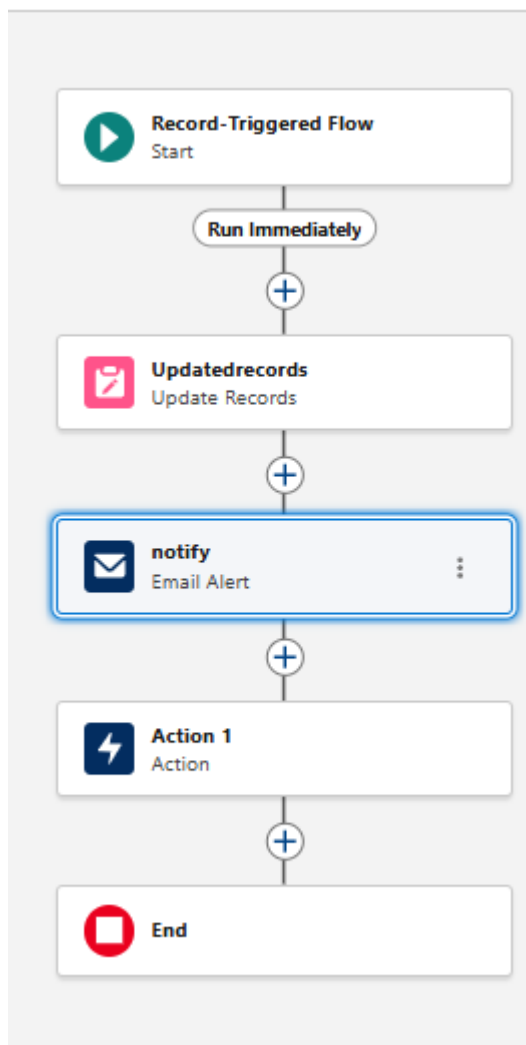
```
trigger CandidateCDCTrigger on Candidate__ChangeEvent (after insert) {
    for(Candidate__ChangeEvent event : Trigger.new){ System.debug('CDC Event
    Received: ' + event.Candidate__c); System.debug('Change Type: ' +
    event.ChangeEventHeader.changeType); // Example: Create a log record
    CandidateLog__c log = new CandidateLog__c( CandidateName__c = event.Name,
    Email__c = event.Email__c, Skills__c = event.Skill__c, Action__c =
    String.valueOf(event.ChangeEventHeader.changeType) ); insert log; } }
```

Key Points:

- `Candidate__ChangeEvent` is the **CDC object** corresponding to your custom object `Candidate__c`.
 - `ChangeEventHeader.changeType` tells you whether it's **CREATE, UPDATE, DELETE, or UNDELETE**.
 - `ChangeEventHeader.recordIds` gives the ID of the changed record.
-

C) Subscribe to CDC in Flows

1. Create a **New Flow** → **Record-Triggered Flow** → **Platform Event Trigger**
 2. Choose **Candidate__ChangeEvent**
 3. Add **actions** like creating a log record, calling Apex, or sending notifications.
- This is low-code and doesn't require Apex if you prefer Flow-based automation.



D) Subscribe to CDC in External Systems

- CDC events can be consumed **outside Salesforce** using the **CometD/Streaming API**.
 - Example use case: When a candidate is updated in Salesforce, an external HR system receives the event immediately.
-

E) Test Change Data Capture

1. Insert or update a **Candidate__c** record
 2. Check **Debug Logs** → **CDC trigger fired**
 3. Verify **CandidateLog__c** or other actions triggered by CDC
-

F) Practical Use in our Portal

- Whenever **candidates, jobs, or skills** are updated externally or internally:
- REST/SOAP Callouts → Salesforce → Publish Platform Event → Update other objects
- CDC → Real-time event → Trigger automation in portal or external systems
- Ensures your **Skill Development Portal stays in sync** in real-time.

Purpose in Project:

- Automatically sync candidate profile updates to external HR or training systems.
- Ensure external systems always have the latest job and skill data.

Example Use Case:

- When a **Job__c** record is updated, CDC pushes the update to external job boards.
-

7. Salesforce Connect

- Salesforce Connect allows external data to appear in Salesforce without storing it in the platform.

Purpose in Project:

- Show live job vacancies from partner portals directly in Salesforce.
- Link external candidate skill certifications to `Candidate__c` without duplication.

Implementation Steps:

1. Define an **External Data Source**.
2. Map external objects to Salesforce.
3. Use **External Lookup Relationships** in relevant objects.

Salesforce Connect allows you to **access external data in real-time** without storing it in Salesforce. You can view and interact with external objects as if they were native Salesforce objects.

A) Key Concepts

Term	Description
External Object	Represents a table in an external system. Similar to a custom object but data resides externally.
External Data Source	Connection settings to the external system (OData, Salesforce, etc.).
OData	Protocol supported by Salesforce Connect to read/write data from external sources.

B) Setup Salesforce Connect

1. **Create External Data Source**
 - Setup → External Data Sources → New External Data Source



SETUP

External Data Sources

New External Data Source

[Help for this Page](#) ?

Connect to another Salesforce org or a third-party database or content system.

[Save](#) [Save and New](#) [Cancel](#)External Data Source Name Type

Parameters

URL Connection Timeout (Seconds) Writable External Objects ☐High Data Volume ☐Server Driven Pagination ☐Request Row Counts ☒Compress Requests ☐Enable Search ☒Use Free-Text Search Expressions ☐Format Special Compatibility Display Server Errors ☐Eligible for External Change Data Capture ☐

Authentication

Certificate Identity Type Authentication Protocol [Save](#) [Save and New](#) [Cancel](#)

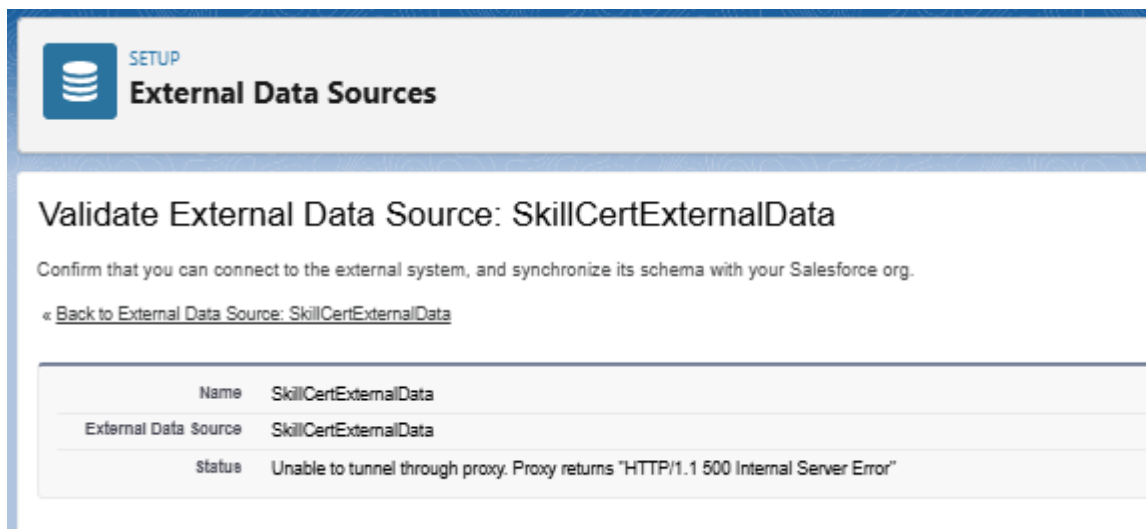
- Example:

Field	Value
Data Source Name	SkillCertExternalData
Type	OData 4.0
URL	https://api.skillcert.com/odata (replace with your endpoint)
Authentication	Named Credential → SkillCertAPI_Test

Field	Value
Identity Type	Named Principal
Save → Validate and Sync	

2. Validate and Sync External Objects

- Click **Validate and Sync**
- Salesforce fetches metadata from the external system
- Select objects like `Candidates`, `Jobs`, `Skills` → Sync



3. External Objects in Salesforce

- These appear under **External Objects** in Object Manager
- You can create **list views, reports, and relationships** like custom objects

C) Access External Data

- **Tabular View:** Add External Object tabs → Users can see external data in Salesforce UI

- **Apex Access:**

```
List<Candidate__x> candidates = [SELECT Name, Email__c, Skill__c FROM
Candidate__x LIMIT 10]; System.debug(candidates);
```

`__x` suffix indicates **external object**



SETUP

External Objects

External Object

Data

[Standard Fields \(2\)](#) | [Custom Fields & Relationships \(0\)](#) | [Page Layouts \(1\)](#) | [Field Sets \(0\)](#) | [Compact Layouts \(1\)](#) | [Buttons, Links, and Actions \(6\)](#)

External Object Definition

Detail

Edit

Delete

Singular Label	Data	Description	
Plural Label	Datas	Enable Reports	☒
Object Name	Data	Name Field	External ID
API Name	Data__x	Deployment Status	Deployed
External Data Source	SkillCertExternalData	Allow Search	☐ If selected, also enable search on the external data source.
Table Name	Candidate		
Display URL Reference Field			
Created By	T. Rupasree , 9/23/2025, 9:14 AM	Modified By	T. Rupasree , 9/23/2025, 9:14 AM

Standard Fields

Action	Field Label	Field Name	Data Type
Display URL		DisplayUrl	URL(1000)
External ID		ExternalId	External Lookup

Custom Fields & Relationships

New

No custom fields defined

Page Layouts

New

Page Layout Assignment

Action	Page Layout Name	Created By	Modified By
Edit Del	Data Layout	T. Rupasree , 9/23/2025, 9:14 AM	T. Rupasree , 9/23/2025, 9:14 AM

Field Sets

New

No field sets defined.

Compact Layouts

New

Compact Layout Assignment

Action	Label	API Name	Primary	Modified By
--------	-------	----------	---------	-------------

D) Practical Use in Your Portal

- Your **Skill Development Portal** can display external candidates, jobs, and skills in **real-time** without storing them in Salesforce.
- Combine with **Flows or Apex** to perform actions on external data.

E) Key Points

1. **Read/Write** depends on the external system and permissions
 2. **No storage limits** in Salesforce — data remains external
 3. **Real-time access** ensures portal always shows latest data
-

8. API Limits

Salesforce enforces limits on API calls to prevent abuse and maintain performance.

Project Relevance:

- Ensure external integrations like skill certification API or job board API do not exceed daily limits.
- Example: 15,000 calls/day in Enterprise Edition.

Best Practices:

- Use **batching** and **Platform Events** to minimize API usage.
- Monitor usage via **Setup → API Usage**.

API limits control how many API calls your org can make in a 24-hour period.

Every **REST/SOAP callout, Apex callout, Salesforce Connect request, or external system request** counts toward limits.

A) Why API Limits Matter

- Avoid hitting limits → your integrations fail temporarily
 - Keep your **REST/SOAP callouts and CDC/Platform Event usage optimized**
 - Salesforce enforces different limits for **Enterprise, Developer, and Unlimited Editions**
-

B) Types of API Calls Counting Toward Limits

API Call	Counts toward limit?
REST API callouts	✓
SOAP API callouts	✓
Bulk API	✓
Streaming API (CDC / Platform Events)	✓
Salesforce Connect via OData	✓
Tooling API / Metadata API	✓

C) Check API Usage

1. System Overview

- Setup → System Overview → Check API Requests Used



SETUP

System Overview

System Overview

View key usage data for your org.

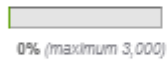
[Help for this Page](#)[Configure Messages](#)

Schema

YOUR CUSTOM OBJECTS + YOUR CUSTOM SETTINGS ⓘ

12

TOTAL CUSTOM OBJECTS + TOTAL CUSTOM SETTINGS ⓘ

14

YOUR CUSTOM METADATA TYPES ⓘ

0

TOTAL CUSTOM METADATA TYPES ⓘ

0

CUSTOM METADATA TYPE USAGE ⓘ

0%

DATA STORAGE ⓘ

563 KB (Approx.)

API Usage

API REQUESTS, LAST 24 HOURS

187

1% (maximum 15,000)

2. API Usage Logs

- Setup → API Usage Notifications
- Setup → Debug Logs → Filter by API

- Custom Code
- Lightning Components
- Debug Mode**
- Environments
- Logs
- Debug Logs**

Didn't find what you're looking for?
Try using Global Search.

SETUP Debug Logs

Debug Logs

[Help for this Page ?](#)

A debug log records database operations, system processes, and errors that occur when executing a transaction or while running unit tests. The system generates a debug log for a user every time that user executes a transaction and the user has a trace flag with start and expiration dates that contain the transaction's start time. You can monitor and retain debug logs for the users specified below. One SFDC_DevConsole debug level is shared by all DEVELOPER_LOG trace flags in your org.

View: All ▼ [Create New View](#)

User Trace Flags		New				
Action	Name ↑	Log Type	Requested By	Start Date	Expiration Date	Debug Level Name
Delete Edit Filters	Executive_Eric	DEVELOPER_LOG	Eric Executive	4/7/2022, 6:24 AM	4/7/2022, 6:29 AM	SFDC_DevConsole
Delete Edit Filters	Rupasree_T.	DEVELOPER_LOG	T. Rupasree	9/23/2025, 7:26 AM	9/23/2025, 8:36 AM	SFDC_DevConsole

[Previous Page](#) | [Next Page](#)

Debug Logs

Delete All

User	Request Type	Application	Operation	Status	Duration (ms)	Log Size (bytes)	Start Time
------	--------------	-------------	-----------	--------	---------------	------------------	------------

Delete All

3. Apex Code Check

```
Limits.getLimitCallouts(); // Total allowed callouts Limits.getCallouts();
// Callouts used in current transaction
```

SkillManager.apxc *
JobApplicationHandler.apxc *
SkillManagerSafe.apxc *
Log executeAnonymous @ 9/21/2025, 5:07:56 PM *
Log executeAnonymous @ 9/23/2025, 5:25:35 PM *
Log executeAnonymous @ 9/23/2025, 5:25:35 PM *

Execution Log

Timestamp	Event	Details
21:50:30:001	USER_INFO	[EXTERNAL]005gl.0000050p8H[rupasree287375@ageantforce.com][(GMT-07:00) Pacific Daylight Time (America/Los_Angeles)]GMT-07:00
21:50:30:001	EXECUTION_ST...	
21:50:30:001	CODE_UNIT_ST...	[EXTERNAL]execute_anonymous_apex
21:50:30:001	HEAP_ALLOCATE	[95]]Bytes:3
21:50:30:001	HEAP_ALLOCATE	[100]]Bytes:152
21:50:30:001	HEAP_ALLOCATE	[417]]Bytes:408
21:50:30:001	HEAP_ALLOCATE	[430]]Bytes:408
21:50:30:001	HEAP_ALLOCATE	[317]]Bytes:6
21:50:30:001	HEAP_ALLOCATE	[EXTERNAL]]Bytes:3
21:50:30:001	STATEMENT_EX...	[1]
21:50:30:001	STATEMENT_EX...	[1]
21:50:30:001	HEAP_ALLOCATE	[68]]Bytes:5
21:50:30:001	HEAP_ALLOCATE	[74]]Bytes:5
21:50:30:001	HEAP_ALLOCATE	[82]]Bytes:7
21:50:30:001	STATEMENT_EX...	[2]
21:50:30:002	CUMULATIVE_L...	
21:50:30:002	LIMIT_USAGE_...	(default)
21:50:30:000	LIMIT_USAGE_...	Number of SOQL queries: 0 out of 100
21:50:30:000	LIMIT_USAGE_...	Number of query rows: 0 out of 50000
21:50:30:000	LIMIT_USAGE_...	Number of SOSL queries: 0 out of 20
21:50:30:000	LIMIT_USAGE_...	Number of DML statements: 0 out of 150
21:50:30:000	LIMIT_USAGE_...	Number of Publish Immediate DML: 0 out of 150
21:50:30:000	LIMIT_USAGE_...	Number of DML rows: 0 out of 10000
21:50:30:000	LIMIT_USAGE_...	Maximum CPU time: 0 out of 10000
21:50:30:000	LIMIT_USAGE_...	Maximum heap size: 0 out of 6000000
21:50:30:000	LIMIT_USAGE_...	Number of callouts: 0 out of 100
21:50:30:000	LIMIT_USAGE_...	Number of Email Invocations: 0 out of 10
21:50:30:000	LIMIT_USAGE_...	Number of future calls: 0 out of 50
21:50:30:000	LIMIT_USAGE_...	Number of queueable jobs added to the queue: 0 out of 50
21:50:30:000	LIMIT_USAGE_...	Number of Mobile Apex push calls: 0 out of 10
21:50:30:000	LIMIT_USAGE_...	
21:50:30:002	CUMULATIVE_L...	
21:50:30:003	CODE_UNIT_ST...	

☐ This Frame
☐ Executable
☐ Debug Only
☐ Filter
[Click here to filter the log](#)

Logs
Tests
Checkpoints
Query Editor
View State
Progress
Problems

Status

Test Run

Enqueued Time

Duration

Example in **REST** callout Apex:

```
System.debug('Total allowed callouts: ' + Limits.getLimitCallouts());
System.debug('Callouts used: ' + Limits.getCallouts());
```

Timestamp	Event	Details
21:51:11:001	USER_INFO	[EXTERNAL]3005a.0000002g8H@apexrc387375@apexrcfarc.com(0MT-07:00) Pacific Daylight Time (America/Los_Angeles)(0MT-07:00)
21:51:11:001	EXECUTION_STARTED	
21:51:11:001	CODE_UNIT_STARTED	[EXTERNAL]@execute_anonymous_apex
21:51:11:001	HEAP_ALLOCATE	[90]Bytes:3
21:51:11:001	HEAP_ALLOCATE	[100]Bytes:152
21:51:11:001	HEAP_ALLOCATE	[417]Bytes:408
21:51:11:001	HEAP_ALLOCATE	[430]Bytes:408
21:51:11:001	HEAP_ALLOCATE	[317]Bytes:6
21:51:11:001	HEAP_ALLOCATE	[EXTERNAL]@Bytes:3
21:51:11:001	STATEMENT_EXECUTED	[1]
21:51:11:001	STATEMENT_EXECUTED	[1]
21:51:11:001	HEAP_ALLOCATE	[13]Bytes:24
21:51:11:001	HEAP_ALLOCATE	[60]Bytes:5
21:51:11:001	HEAP_ALLOCATE	[740]Bytes:5
21:51:11:001	HEAP_ALLOCATE	[82]Bytes:7
21:51:11:001	HEAP_ALLOCATE	[13]Bytes:3
21:51:11:001	HEAP_ALLOCATE	[13]Bytes:27
21:51:11:001	USER_DEBUG	[13]DEBUG:Total allowed callouts: 100
21:51:11:001	STATEMENT_EXECUTED	[2]
21:51:11:001	HEAP_ALLOCATE	[23]Bytes:15
21:51:11:001	HEAP_ALLOCATE	[23]Bytes:1
21:51:11:001	HEAP_ALLOCATE	[23]Bytes:15
21:51:11:001	USER_DEBUG	[23]DEBUG:Callouts used: 0
21:51:11:001	CUMULATIVE_LIMIT_USAGE	(default)
21:51:11:000	LIMIT_USAGE	Number of SQL queries: 0 out of 100
21:51:11:000	LIMIT_USAGE	Number of query rows: 0 out of 50000
21:51:11:000	LIMIT_USAGE	Number of SOQL queries: 0 out of 20
21:51:11:000	LIMIT_USAGE	Number of DML statements: 0 out of 150
21:51:11:000	LIMIT_USAGE	Number of Publish Immediate DML: 0 out of 150
21:51:11:000	LIMIT_USAGE	Number of DML rows: 0 out of 10000
21:51:11:000	LIMIT_USAGE	Maximum CPU time: 0 out of 10000

D) Best Practices to Avoid Hitting Limits

- Batch Callouts**
 - Combine multiple REST/SOAP calls in one request if API allows
- Use Named Credentials**
 - Reduces repeated authentication overhead
- Asynchronous Processing**
 - Use `@future(callout=true)`, Queueable, or Batch Apex
- Platform Events / CDC**
 - Prefer events instead of polling external APIs frequently
- Monitor Usage**
 - Use **Scheduled Apex** to log API usage
 - Alert admin when usage approaches 75% of limit

E) Example: Monitor API Usage in Apex

```
public class APILimitMonitor { public static void checkLimits() {
    System.debug('Total Callouts Allowed: ' + Limits.getLimitCallouts());
    System.debug('Callouts Used in this transaction: ' + Limits.getCallouts());
} }
```

- You can schedule this **daily** to ensure your portal doesn't hit limits.

9. OAuth & Authentication

OAuth is a secure authorization protocol to access external systems without sharing credentials.

OAuth (Open Authorization) is a secure protocol that allows your Salesforce org to **access external APIs without storing user passwords**. It's widely used for REST/SOAP integrations and Salesforce Connect.

A) Why OAuth Matters

- Secure authentication without exposing credentials
 - Supports token-based access (Access Token / Refresh Token)
 - Works with **Named Credentials**, which simplifies callouts
-

B) OAuth Flow Types

OAuth Flow	Use Case
Username-Password Flow	Simple server-to-server integration (use cautiously)
Web Server Flow	For apps requiring user login and consent
JWT Bearer Token Flow	Server-to-server, no interactive login
Refresh Token Flow	Automatically refresh access token without user login

For your **Skill Development Portal**, **JWT or Named Credential with OAuth** is recommended for secure REST/SOAP callouts.



SETUP > NAMED CREDENTIALS

SkillCertAPI_OAuth

EditDelete

Label

SkillCertAPI_OAuth

Name

SkillCertAPI_OAuth

URL

https://api.skillcert.com

Enabled for Callouts

☒

Authentication

External Credential

SkillCertExtCred

Client Certificate

Callout Options

Generate Authorization Header ?

☒

Allow Formulas in HTTP Header ?

☒

Allow Formulas in HTTP Body ?

☒

Outbound Network Connection ?

Managed Package Access

Created By Namespace ?

Allowed Namespaces for Callouts ?

Custom Headers

New

C) Configure OAuth in Named Credential

- Go to **Setup** → **Named Credentials** → **New**
- Example configuration:

Field	Value
Label	SkillCertAPI_OAuth

Field	Value
URL	<code>https://api.skillcert.com</code>
Identity Type	Named Principal
Authentication Protocol	OAuth 2.0
Authentication Provider	Create new (if not already)
Scope	<code>read write</code> (as required by API)
Start Authentication Flow on Save Checked	

3. **Create Authentication Provider** (if using OAuth)
 - Setup → Auth. Providers → New
 - Provider Type: Custom or Web Server
 - Consumer Key & Secret: Provided by external system
 - Callback URL: Generated by Salesforce
4. Save Named Credential → Salesforce handles **OAuth token management** automatically

SETUP

Auth. Providers

Auth. Provider

[Help for this](#)

Auth. Provider Detail

Edit

Delete

Clone

Auth. Provider ID	0SOgL000000rpu1
Provider Type	Salesforce
Name	SkillCertAuth
URL Suffix	SkillCertAuth
Consumer Key	
Consumer Secret	
Authorize Endpoint URL	https://login.salesforce.com/services/oauth2/authorize
Token Endpoint URL	https://login.salesforce.com/services/oauth2/token
Use Proof Key for Code Exchange (PKCE) Extension	☒
Default Scope	
Include Consumer Secret In SOAP API Responses	☒
Custom Error URL	
Custom Logout URL	
Registration Handler Type	None
Portal	
Icon URL	
Use Salesforce MFA for this SSO Provider	☐

Salesforce Configuration

Test-Only Initialization URL	https://orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/services/auth/test/SkillCertAuth
Existing User Linking URL	https://orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/services/auth/link/SkillCertAuth
OAuth-Only Initialization URL	https://orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/services/auth/oauth/SkillCertAuth
Single Logout URL	https://orgfarm-dc1cedaabd-dev-ed.develop.my.salesforce.com/services/auth/oidc/logout

Edit

Delete

Clone

D) Use OAuth Named Credential in Apex Callouts

```
Http http = new Http();
HttpRequest request = new HttpRequest();
request.setEndpoint('callout:SkillCertAPI_OAuth/candidates');
request.setMethod('GET');
request.setHeader('Content-Type', 'application/json');
HttpResponse response = http.send(request);
System.debug('OAuth REST Response: ' + response.getBody());
```

Salesforce automatically adds the **Authorization header** with the access token.

Timestamp	Event	Details
21:58:12:007	USER_DEBUG	{@DEBUG}OAuth REST Response: <DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" "http://www.w3.org/TR/html4/strict.dtd">
21:58:12:000	USER_DEBUG	<html><head>
21:58:12:000	USER_DEBUG	<meta type="copyright" content="Copyright (C) 1996-2023 The Squid Software Foundation and contributors">
21:58:12:000	USER_DEBUG	<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
21:58:12:000	USER_DEBUG	<title>ERROR: The requested URL could not be retrieved</title>
21:58:12:000	USER_DEBUG	<style type="text/css"> <!--
21:58:12:000	USER_DEBUG	/*
21:58:12:000	USER_DEBUG	* Copyright (C) 1996-2023 The Squid Software Foundation and contributors
21:58:12:000	USER_DEBUG	*
21:58:12:000	USER_DEBUG	* Squid software is distributed under GNU2+ license and includes
21:58:12:000	USER_DEBUG	* contributions from numerous individuals and organizations.
21:58:12:000	USER_DEBUG	* Please see the COPYING and CONTRIBUTORS files for details.
21:58:12:000	USER_DEBUG	*/
21:58:12:000	USER_DEBUG	/*
21:58:12:000	USER_DEBUG	/*
21:58:12:000	USER_DEBUG	Stylesheet for Squid Error pages
21:58:12:000	USER_DEBUG	Adapted from design by Free CSS Templates
21:58:12:000	USER_DEBUG	http://www.freecsstemplates.org
21:58:12:000	USER_DEBUG	Released for free under a Creative Commons Attribution 2.5 License
21:58:12:000	USER_DEBUG	*/
21:58:12:000	USER_DEBUG	/* Page basics */
21:58:12:000	USER_DEBUG	{
21:58:12:000	USER_DEBUG	font-family: verdana, sans-serif;
21:58:12:000	USER_DEBUG	}
21:58:12:000	USER_DEBUG	html body {
21:58:12:000	USER_DEBUG	margin: 0;
21:58:12:000	USER_DEBUG	padding: 0;
21:58:12:000	USER_DEBUG	background: #f9f9f9;
21:58:12:000	USER_DEBUG	font-size: 12px;

E) Refresh Tokens (Optional)

- Named Credentials manage refresh tokens automatically
- If using **manual Apex**, you can refresh the token via OAuth endpoint:

```
// Example: POST to token endpoint with refresh_token, client_id, client_secret
```

Usually unnecessary if using **Named Credentials**.

F) Security Best Practices

1. Never hardcode client secrets or passwords in Apex
2. Use **Named Credentials + OAuth** whenever possible
3. Limit **scope** to only what your portal needs (`read`, `write`)
4. Audit **API usage** regularly to avoid abuse

Project Implementation:

- Connect to external training or job portals securely.
- Use **OAuth 2.0 Named Credentials** for REST API authentication.

Example Flow:

1. Portal requests access token via OAuth.
2. Salesforce stores token in Named Credential.
3. Use token in API callouts automatically.

10. Remote Site Settings

Definition:

Salesforce requires all external endpoints to be whitelisted in **Remote Site Settings** for security.

Remote Site Settings allow Salesforce to **connect to external URLs**. Any REST/SOAP callout to an external system must either use:

- **Named Credentials** (recommended) – no separate Remote Site required
- **Or Remote Site Settings** if not using Named Credentials

A) Add a Remote Site

1. Go to **Setup** → **Security** → **Remote Site Settings** → **New Remote Site**
2. Example configuration:

Field	Value
Remote Site Name	SkillCertAPI
Remote Site URL	<code>https://api.skillcert.com</code>
Description	REST/SOAP endpoints for Skill Certification API
Active	Checked

3. Save → Salesforce can now make **callouts to this URL**

Remote Site Edit

Enter the URL for the remote site. All s-controls, JavaScript OnClick commands in custom buttons, Apex, and AJAX proxy calls use this Web address from salesforce.com.

Remote Site Edit Save Save & New Cancel

Remote Site Name SkillCertAPI

Remote Site URL https://api.skillcert.com

Disable Protocol Security ☐ ⓘ

Description

Active ☒

Save Save & New Cancel

B) Best Practices

1. **Use Named Credentials whenever possible** – handles authentication and headers automatically
2. **Limit URLs** – only add domains required by your portal
3. **Secure HTTPS URLs** – Salesforce requires SSL endpoints for callouts

C) Test Callouts After Remote Site Setting

1. Go to **Developer Console** → **Execute Anonymous**:

```
Http http = new Http(); HttpRequest req = new HttpRequest();  
req.setEndpoint('https://api.skillcert.com/candidates'); // or  
callout:SkillCertAPI_Test if Named Credential req.setMethod('GET');  
HttpResponse res = http.send(req); System.debug(res.getBody());
```

Enter Apex Code

```

1 // Run this in Execute Anonymous Window
2 Http http = new Http();
3 HttpRequest request = new HttpRequest();
4
5 // Use a public test API endpoint first (safe, no auth needed)
6 request.setEndpoint('https://jsonplaceholder.typicode.com/posts/1');
7 request.setMethod('GET');
8
9 // Set headers if needed
10 request.setHeader('Content-Type', 'application/json');
11
12 try {
13     HttpResponse response = http.send(request);
14
15     System.debug('Status Code: ' + response.getStatusCode());
16     System.debug('Response Body: ' + response.getBody());
17 } catch (System.CalloutException e) {
18     System.debug('Callout failed: ' + e.getMessage());
19 }
20

```

☒ Open Log
 Execute
 Execute Highlighted

- If successful → Salesforce can reach external endpoints
- If error: `System.CalloutException: You have not defined a remote site setting for this URL` → add Remote Site

File • Edit • Debug • Test • Workspace • Help

Log executeAnonymous 09/23/2025, 5:07:58 PM • Log executeAnonymous 09/23/2025, 5:25:35 PM • Log executeAnonymous 09/23/2025, 6:27:27 PM • Log executeAnonymous 09/23/2025, 6:30:51 PM • Log executeAnonymous 09/23/2025, 10:05:29 PM

Execution Log

Timestamp	Event	Details
22:05:29:007	EXCEPTION_THROWN	[13]System.CalloutException: Unauthorized endpoint, please check Setup->Security->Remote site settings. endpoint = https://jsonplaceholder.typicode.com/posts/1
22:05:29:007	HEAP_ALLOCATE	[13]@N/A:134
22:05:29:007	HEAP_ALLOCATE	[80]@N/A:5
22:05:29:007	HEAP_ALLOCATE	[74]@N/A:5
22:05:29:007	HEAP_ALLOCATE	[82]@N/A:7
22:05:29:007	VARIABLE_SCOPE	[18]@System.CalloutException:true/false
22:05:29:007	VARIABLE_ASSIGN	[18]@System.CalloutException:true/false
22:05:29:007	STATEMENT_EXECUTE	[18]@System.CalloutException:true/false
22:05:29:007	STATEMENT_EXECUTE	[18]@System.CalloutException:true/false
22:05:29:007	HEAP_ALLOCATE	[18]@N/A:16
22:05:29:007	HEAP_ALLOCATE	[18]@N/A:130
22:05:29:007	HEAP_ALLOCATE	[18]@N/A:146
22:05:29:007	USER_DEBUG	[18]@System.CalloutException:true/false
22:05:29:007	CUMULATIVE_LIMIT_USAGE	
22:05:29:007	LIMIT_USAGE	(default)
22:05:29:000	LIMIT_USAGE	Number of SQL queries: 0 out of 100
22:05:29:000	LIMIT_USAGE	Number of query rows: 0 out of 50000
22:05:29:000	LIMIT_USAGE	Number of SQL queries: 0 out of 20
22:05:29:000	LIMIT_USAGE	Number of DML statements: 0 out of 150
22:05:29:000	LIMIT_USAGE	Number of DML statements: 0 out of 150
22:05:29:000	LIMIT_USAGE	Number of DML rows: 0 out of 10000
22:05:29:000	LIMIT_USAGE	Maximum CPU time: 0 out of 10000
22:05:29:000	LIMIT_USAGE	Maximum heap size: 0 out of 6000000
22:05:29:000	LIMIT_USAGE	Number of callouts: 1 out of 100
22:05:29:000	LIMIT_USAGE	Number of email invocations: 0 out of 10
22:05:29:000	LIMIT_USAGE	Number of future calls: 0 out of 50
22:05:29:000	LIMIT_USAGE	Number of queueable jobs added to the queue: 0 out of 50
22:05:29:000	LIMIT_USAGE	Number of Mobile Apex push calls: 0 out of 10
22:05:29:000	LIMIT_USAGE	
22:05:29:007	CUMULATIVE_LIMIT_USAGE	
22:05:29:007	CODE_UNIT_FINISHED	execute_anonymous_apex
22:05:29:008	EXECUTION_FINISHED	

This Frame

☒ Executable
 ☐ Debug Only

☐ Filter
 Click here to filter the log

Logs

Tests

Checkpoints

Query Editor

View State

Progress

Problems

Status

Test Run

Elapsed Time

Duration

Failures

Total

Overall Code Coverage

Class	Percent	Lines
Overall	0%	
AssignSkillToTraining	0%	0/1
AssignSkillBatch	0%	0/11
AssignSkillQueueable	0%	0/9

D) Practical Tip for Your Portal

- For **Skill Development & Employment Portal**, every external REST/SOAP API endpoint (like SkillCert API) must be either:
 - Configured in a Named Credential** → preferred
 - Or added in Remote Site Settings** → fallback

- This ensures **all callouts for candidate/job/skill syncing work** without security errors.

Project Implementation:

- Add URLs like:
 - `https://api.skillcert.com`
 - `https://externaljobportal.com`
 - Ensures Apex callouts succeed without security errors.
-

11. Integration Workflow – Example for Project

Scenario: Candidate completes a new skill certification, and we want to update portal and external systems automatically.

1. Candidate passes skill test → `Skill__c` record created.
 2. **Platform Event** `CandidateSkillUpdated__e` is published.
 3. External systems subscribed to event fetch the data via **Change Data Capture**.
 4. Apex callout to external skill API verifies certificate using **Named Credential**.
 5. Job boards updated using **REST API callout** with OAuth authentication.
 6. All external endpoints validated in **Remote Site Settings**.
-

12. Security Considerations

- Use **Named Credentials** to avoid hardcoding credentials.
 - Limit external API access using **OAuth scopes**.
 - Ensure callouts handle **timeouts and errors** gracefully.
 - Monitor API usage to avoid exceeding **API limits**.
-

Summary Table

Feature	Purpose in Project	Key Objects / Records
Named Credentials	Secure API access	Skill__c, Job__c
External Services	Declarative integration	Candidate__c
Web Services (REST/SOAP)	Push/pull data from external portals	Job__c, Skill__c
Callouts	Trigger external actions	Candidate__c, Job__c
Platform Events	Real-time notifications	CandidateSkillUpdated__e
Change Data Capture	Real-time record sync	Job__c, Candidate__c
Salesforce Connect	Access external data without duplication	Job__c, Candidate__c
API Limits	Prevent system overload	All integrated APIs
OAuth & Authentication	Secure connection to external services	Job Boards, Skill APIs
Remote Site Settings	Whitelist endpoints for callouts	All external endpoints

This Phase 7 ties with our **previous objects and flows** and provides practical examples relevant to our **Skill Development and Employment Portal Project**.